

图的表示与遍历

关系怎样被表示，并从起点覆盖而不重复失控

408 · 数据结构 Topic DS-07

母问题：图的关系如何变成可验证的访问过程？

图遍历的生成链是

Vertices/Edges → Representation → Frontier → Visited State → Expansion → Coverage.

邻接矩阵用空间换常数时间的边查询，邻接表按实际边数保存关系；DFS 与 BFS 都要维护 visited，区别在 frontier 的 LIFO/FIFO 取出规则。遍历只负责覆盖和发现关系，不自动解决最短路、MST 或拓扑目标。

本册定位与 Owner 边界

- **Owns**：图、邻接矩阵/邻接表/十字链表/邻接多重表、方向与重边语义、DFS/BFS、visited、连通分量与遍历森林。
- **Uses**：DS02 的栈/队列端点契约和 DS-B01 frontier 接口；DS08 使用本册表示。
- **Stops**：最短路、最小生成树、拓扑排序等全局目标归 DS08；网络路由只 Use。

目录

1	表示先于算法	2
1.1	十字链表：一条有向弧同时进入出边链与入边链	2
1.2	邻接多重表：无向边是共享对象，而不是两份边副本	2
2	visited 是覆盖不变量	2
3	DFS、BFS 与遍历森林	2
3.1	四个动词固定遍历状态	3
4	代码与测试证据	3
5	边界与概念分离	5
6	做题控制协议	6

1 表示先于算法

邻接矩阵 $A[i][j]$ 直接记录边存在性或权值，适合稠密图和频繁边查询，空间 $\Theta(V^2)$ 。邻接表为每个顶点保存出边列表，空间 $\Theta(V + E)$ ，遍历一个顶点的邻居只扫描实际出边。无向边需要在两端各登记一次；有向边只登记出发端。平行边和自环是否允许，必须在接口中先固定。

1.1 十字链表：一条有向弧同时进入出边链与入边链

普通有向邻接表让枚举顶点 u 的出边很自然，但若枚举入边，往往还需逆邻接表。十字链表把一条弧记录为

$$(tail, head, tlink, hlink, info),$$

其中 $tlink$ 连接同弧尾的下一条出边， $hlink$ 连接同弧头的下一条入边；顶点记录同时保存首条出弧和首条入弧。一个边对象被两条链引用，但只存一份权值和附加信息。

不变量是：每条弧在其尾顶点的出边链中出现一次，也在其头顶点的入边链中出现一次；删除弧必须同时从两条链摘除。它适合同时需要入度、出度和双向邻接扫描的有向图。

1.2 邻接多重表：无向边是共享对象，而不是两份边副本

无向邻接表通常把边 $\{u, v\}$ 分别登记在 u, v 的链中。邻接多重表为每条无向边只建一个边节点，保存两个端点 $ivex, jvex$ 以及分别连接两个端点下一条关联边的 $ilink, jlink$ 。从顶点 u 扫描时，要根据 u 是该边的哪一个端点选择对应 $link$ 。

其收益是边级插删与标记只作用于一个共享对象；代价是遍历 $link$ 时必须判断当前端点角色。十字链表面向有向弧的入/出双链，邻接多重表面向无向边的两端关联，二者不能只凭“都有两条链”互换。

表示选择的题面信号

只需判断任意两点是否相邻且图较稠密，优先邻接矩阵；按实际边枚举出边，邻接表足够；有向图同时频繁枚举入边与出边，看十字链表；无向图需要对同一边做删除、标记或边级维护，看邻接多重表。选择依据是需要沿哪个关系方向访问，而不是结构名称的新旧。

2 visited 是覆盖不变量

遍历的核心不变量是：每个顶点最多被“首次发现”一次，已发现但未展开的顶点位于 $frontier$ ，展开后其出边已按表示规则检查。非连通图从一个起点只能覆盖一个可达分量；要得到遍历森林，必须在所有顶点上补一层外循环。

同一关系，不同 frontier

从 0 出发，若先把邻居压栈，得到 DFS 的深路径；若把邻居入队，得到 BFS 的层次距离。两者都可以覆盖同一可达集，但只有无权图 BFS 的首次发现层数具有最短边数语义。

3 DFS、BFS 与遍历森林

递归 DFS 隐式使用调用栈；显式 DFS 必须约定邻接顺序，否则输出序列不唯一。BFS 使用队列，入队时就标记 $visited$ ，不能等出队才标记，否则多个前驱会重复入队。对邻接表，二者时间均为 $O(V + E)$ ，额外空间为 $O(V)$ ；矩阵扫描每个顶点所有候选邻居，时间为 $O(V^2)$ 。

3.1 四个动词固定遍历状态

把任何遍历逐行代码都翻译成四个动作：

Discover → Mark → Enqueue/Push → Expand

Discover 是第一次遇到未访问邻居；Mark 把“已发现”写入 visited；Enqueue/Push 把它加入待处理边界；Expand 是将来取出后扫描邻接表。Mark 必须紧跟 Discover，因为 frontier 中的顶点已经被别的路径“预约”，不能再次加入。

为什么出队时标记会重复

设 0 同时连接 1,2，且 1,2 都连接 3。BFS 展开 1 时发现 3 并入队；若尚未标记，展开 2 时又会把 3 入队。发现时标记则把状态从 'Unseen' 立即改为 'Discovered'，第二条边只被检查，不再制造第二份 frontier 项。

顶点状态	所在位置	已经保证什么	下一动作
Unseen	不在 frontier	尚无已知发现路径	被邻边 Discover
Discovered	frontier 中	已有一条发现路径，visited=true	等待取出
Expanded	frontier 外	所有邻边已扫描	不再展开

DFS 与 BFS 共享这三态；区别只在 Discovered 顶点按 LIFO 还是 FIFO 取出。遍历森林再增加外层动作：找最小的 Unseen 顶点作为新根，重复同一状态机。

4 代码与测试证据

完整实现位于 code/ds07_graph_traversal.hpp，测试位于 code/ds07_graph_traversal_test.cpp。

Listing 1: 邻接表、BFS、DFS 与可达覆盖

```
12 class Graph {
13     public:
14         explicit Graph(std::size_t vertices, bool directed = false)
15             : adjacency_(vertices), matrix_(vertices, std::vector<bool>(
16               vertices, false)), directed_(directed) {}
17         std::size_t size() const { return adjacency_.size(); }
18         void add_edge(std::size_t from, std::size_t to) {
19             check(from); check(to);
20             adjacency_[from].push_back(to);
21             matrix_[from][to] = true;
22             if (!directed_) adjacency_[to].push_back(from);
23             if (!directed_) matrix_[to][from] = true;
24         }
25         const std::vector<std::size_t>& neighbors(std::size_t vertex) const {
26             check(vertex); return adjacency_[vertex];
27         }
28         bool has_edge(std::size_t from, std::size_t to) const {
```

```

28     check(from); check(to); return matrix_[from][to];
29 }
30 std::vector<std::size_t> bfs(std::size_t start) const {
31     check(start); std::vector<bool> seen(size()); std::vector<std::size_t>
32     > order;
33     std::queue<std::size_t> frontier; frontier.push(start); seen[start] =
34     true;
35     while (!frontier.empty()) {
36         auto vertex = frontier.front(); frontier.pop(); order.push_back(
37         vertex);
38         for (auto next : adjacency_[vertex]) if (!seen[next]) { seen[next]
39         = true; frontier.push(next); }
40     }
41     return order;
42 }
43 std::vector<std::size_t> dfs(std::size_t start) const {
44     check(start); std::vector<bool> seen(size()); std::vector<std::size_t>
45     > order;
46     std::stack<std::size_t> frontier; frontier.push(start); seen[start] =
47     true;
48     while (!frontier.empty()) {
49         auto vertex = frontier.top(); frontier.pop();
50         order.push_back(vertex);
51         for (auto it = adjacency_[vertex].rbegin(); it != adjacency_[vertex
52         ].rend(); ++it) {
53             if (!seen[*it]) {
54                 seen[*it] = true;
55                 frontier.push(*it);
56             }
57         }
58     }
59     return order;
60 }
61
62 private:
63     std::vector<std::vector<std::size_t>> adjacency_;
64     std::vector<std::vector<bool>> matrix_;
65     bool directed_;
66     void check(std::size_t vertex) const {
67         if (vertex >= size()) throw std::out_of_range("graph vertex");
68     }
69 };
70
71 // namespace ipara::ds07
72
73 #endif

```

Listing 2: 方向、非连通与空图测试

```
8
9 int main() {
10     Graph graph(6);
11     graph.add_edge(0, 1); graph.add_edge(0, 2); graph.add_edge(1, 3);
12     graph.add_edge(2, 4);
13     assert(graph.has_edge(0, 1) && graph.has_edge(1, 0));
14     assert(!graph.has_edge(0, 5));
15     assert((graph.bfs(0) == std::vector<std::size_t>{0, 1, 2, 3, 4}));
16     assert((graph.dfs(0) == std::vector<std::size_t>{0, 1, 3, 2, 4}));
17     assert(graph.bfs(5).size() == 1);
18
19     Graph diamond(4, true);
20     diamond.add_edge(0, 1); diamond.add_edge(0, 2);
21     diamond.add_edge(1, 3); diamond.add_edge(2, 3);
22     assert((diamond.bfs(0) == std::vector<std::size_t>{0, 1, 2, 3}));
23     assert((diamond.dfs(0) == std::vector<std::size_t>{0, 1, 3, 2}));
24
25     Graph directed(2, true); directed.add_edge(0, 1);
26     assert(directed.neighbors(1).empty());
27     assert(directed.has_edge(0, 1) && !directed.has_edge(1, 0));
28     bool threw = false; try { graph.add_edge(6, 0); } catch (const std::
        out_of_range&) { threw = true; }
29     assert(threw);
30 }
```

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds07_graph_traversal_test.cpp -o /tmp/ds07_test
/tmp/ds07_test
```

5 边界与概念分离

操作	邻接表	邻接矩阵	不变量/前提	边界
遍历	$O(V + E)$	$O(V^2)$	每顶点首次发现一次	空图、非连通
边查询	$O(\deg u)$	$O(1)$	方向语义固定	重边/自环
空间	$O(V + E)$	$O(V^2)$	只保存约定关系	稠密/稀疏

概念 A	≠ 概念 B	判据	错因
DFS	≠ BFS	栈 vs 队列	误写距离结论
首次发现	≠ 出队/出栈	visited 时机	重复入 frontier
遍历覆盖	≠ 最短路/MST	目标不同	把结构事实当目标证明

6 做题控制协议

先标顶点、方向、重边和表示；再确定起点与邻接访问顺序。看到“访问所有可达顶点”选 DFS/BFS，看到“最少边数”才升级到 BFS 距离，看到权值优化转交 DS08。每一步写出 visited 与 frontier 的变化，最后检查是否遗漏非连通分量。

压缩复原

五问：边存在哪里？当前 frontier 是栈还是队列？何时标记 visited？起点能覆盖哪些分量？题目要覆盖还是要优化？